

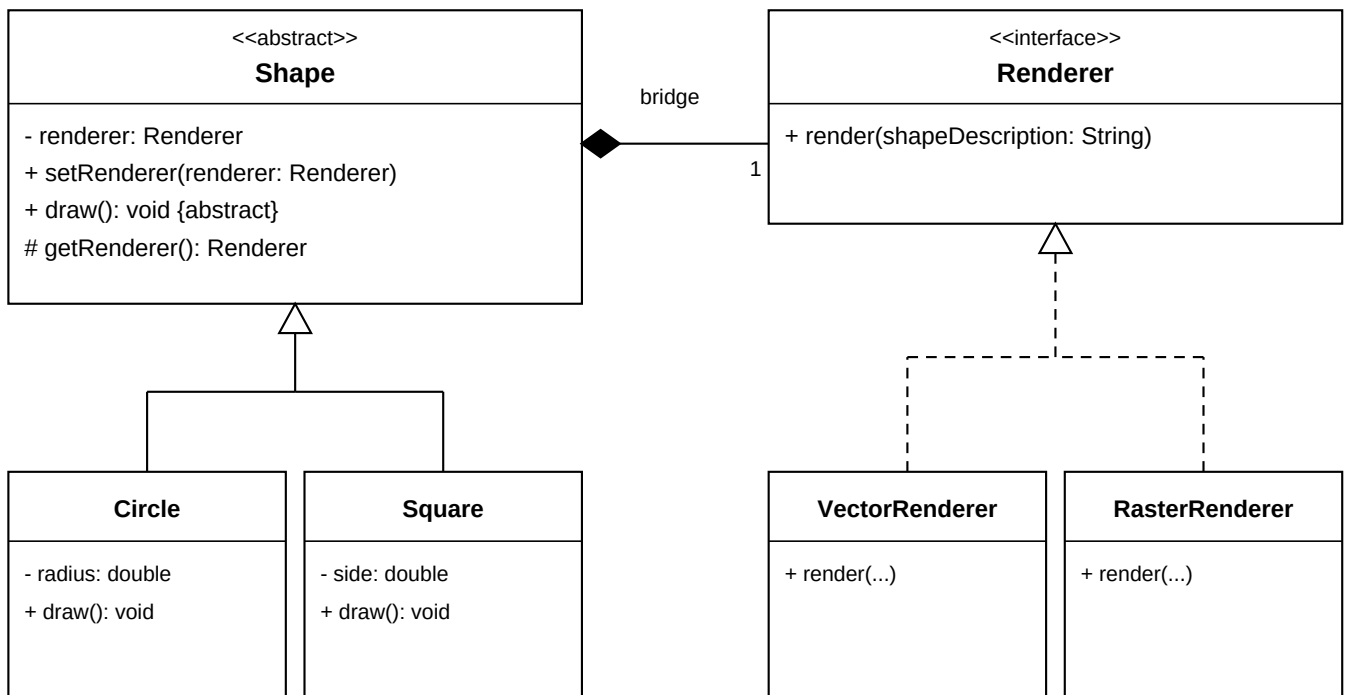
Assignment 3 Bridge Pattern

Introduction

The chosen topic is Shape-Renderer. Circle and Square represent shapes, while VectorRenderer and RasterRenderer represent ways to draw them. Bridge fits because a shape stores a Renderer reference and delegates drawing to it. The shape classes do not depend on a specific renderer class.

Main first draws both shapes with vector renderers, then changes their renderer references and draws the same objects with raster renderers. Drawing is represented by console output. Bridge separates two design dimensions from the start; Adapter instead makes an existing incompatible interface usable.

UML class diagram



Solid line: inheritance Dashed line: interface implementation Hollow triangles point upward
Filled diamond: Shape contains a Renderer reference (object composition)

Shape is the abstraction; Circle and Square are refined abstractions. Renderer is the implementor interface; VectorRenderer and RasterRenderer are concrete implementors. Main is the client that composes and switches these objects.

Clean Code principles

The following excerpts are from the implementation. Each explanation states the design choice and its benefit.

1 Single responsibility

```
getRenderer().render("Circle with radius " + radius);
```

Circle stores its radius and requests drawing. The renderer handles the output, keeping shape data separate from rendering details.

2 Depend on abstractions

```
private Renderer renderer;
```

Shape refers to the Renderer interface, not VectorRenderer or RasterRenderer. Either implementation can be supplied without changing Shape.

3 Encapsulation

```
private final double radius;  
public void setRenderer(Renderer renderer) {  
    this.renderer = renderer;  
}
```

These excerpts are from Circle and Shape respectively. The radius is private and final. Shape keeps its renderer private and exposes setRenderer for runtime switching.

4 Meaningful names

```
void render(String shapeDescription);
```

render names the operation and shapeDescription names its input. Shape, Renderer, Circle, and Square also make the roles clear without comments.

5 Open closed principle

```
Shape circle = new Circle(5, new VectorRenderer());  
circle.setRenderer(new RasterRenderer());
```

A new renderer can implement the same interface without editing Shape, Circle, or Square. Only client construction needs to choose the new renderer.

Conclusion

Bridge adds an interface, extra classes, and delegation, but avoids separate classes for every shape-renderer combination. A new shape can provide its description without changing the renderers; a new renderer can handle descriptions without changing the shapes. The demo uses text output to keep it small. A real graphics application would need a richer rendering contract.

GitHub repository

<https://github.com/asankhanshyngys/assignment-3-bridge-pattern>